PROGRAMACIÓN II

PROYECTO 1 — CUARTO PERIODO ACADÉMICO

MARVEL HEROES

STRATEGO GAME



Grupos de 3 estudiantes · Entrega vía GITHUB A CANVAS

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

1. DESCRIPCIÓN GENERAL DEL PROYECTO

Marvel Heroes – Stratego Game es la versión digital del clásico juego de estrategia Stratego, recreado con los personajes del universo Marvel. La implementación deberá realizarse completamente de forma visual (GUI) utilizando la biblioteca Java Swing. No se aceptará ninguna interacción por consola; todo el flujo del programa deberá manejarse a través de ventanas, paneles, botones y componentes gráficos.

El proyecto pone a prueba la capacidad del estudiante para diseñar e implementar un sistema orientado a objetos no trivial, aplicar correctamente los principios de la Programación Orientada a Objetos (POO) y construir una interfaz gráfica funcional, robusta y estéticamente cuidada.

1.1 Objetivo del Juego

Existen dos bandos enfrentados: el equipo de los HÉROES y el equipo de los VILLANOS. Cada bando posee una pieza especial denominada TIERRA. La condición de victoria es una de las siguientes:

- Capturar la TIERRA del equipo contrario, O
- Destruir TODAS las piezas móviles del equipo contrario.

Las piezas tienen asignado un rango de poder (1 – 10) que determina el resultado de los combates, con excepciones especiales descritas más adelante.

1.2 El Tablero

© UNITEC — Proyecto de Programación II | Todos los derechos reservados



Figura 1: Tablero 10×10 del juego con zonas prohibidas

El tablero es una cuadrícula de 10×10 celdas. Cada bando ocupa inicialmente las primeras 4 filas de su lado (filas 0-3 para Villanos y filas 6-9 para Héroes). En el centro del tablero existen dos zonas prohibidas de 2×2 celdas en las cuales ninguna pieza puede colocarse ni moverse.

1.3 Los Personajes y sus Rangos

Cada bando dispone de 40 fichas con los siguientes rangos:

© UNITEC — Proyecto de Programación II | Todos los derechos reservados



Figura 2: Tabla de personajes de cada bando con sus rangos

Reglas especiales de combate:

- Rango 3 (Scout): únicas piezas capaces de destruir las piezas BOMBA (Nova Blast / Pumpkin Bomb).
- Rango 2 (Mistico/Mística): pueden moverse cualquier número de casillas vacías en línea recta ortogonal (como la torre en ajedrez).
- Rango 1 (Agente/Agente S.H.I.E.L.D): pueden ser derrotadas por cualquier pieza, BUT si ellas atacan a una pieza de Rango 10, las derrota. También puede capturar la TIERRA.
- Piezas BOMBA (Nova Blast / Pumpkin Bomb): no se mueven y destruyen a cualquier pieza que las ataque, EXCEPTO las de Rango 3.
- Piezas TIERRA: no se mueven; capturarla otorga la victoria inmediata.

⚠ **NOTA:** *Las fichas se colocan con la identidad oculta para el adversario. En el proyecto esto se gestiona a través de las modalidades de juego descritas en la sección de Configuración.*

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

2. REGLAS DEL JUEGO

2.1 Turnos

El juego se desarrolla por turnos alternados, comenzando siempre el jugador de los HÉROES. En cada turno el jugador activo puede realizar una de dos acciones: MOVER o ATACAR una pieza. Si el jugador no tiene ningún movimiento posible, deberá rendirse y el juego finaliza.

2.2 Reglas de Movimiento

1. Únicamente se puede mover una (1) pieza por turno.
2. Las piezas se mueven una sola casilla adyacente vacía: adelante, atrás o a los lados (no diagonal). Excepción: piezas de Rango 2.
3. No se puede mover a una casilla ocupada por una pieza propia ni a una zona prohibida.
4. Rango 2 puede desplazarse múltiples casillas en línea recta ortogonal mientras el camino esté despejado.

> ⚠ **NOTA:** *Al mover una pieza de Rango 2 varios espacios, esta se revela automáticamente al oponente ya que se evidencia su comportamiento especial.*

2.3 Reglas de Captura / Ataque

Una pieza (que no sea bomba ni tierra) puede atacar a una pieza adyacente enemiga. Al atacar, ambas identidades se revelan:

5. La pieza de mayor rango vence y permanece en el tablero; la de menor rango es eliminada.
6. Si ambas piezas tienen el mismo rango, ambas son eliminadas.

★ **IMPORTANTE:** Las piezas eliminadas DEBEN mostrarse en todo momento en pantalla (panel de piezas vencidas) para que ambos jugadores puedan construir su estrategia.

Excepciones de combate:

- Rango 1 ATACANDO a Rango 10: Rango 1 gana.
- Cualquier pieza ATACANDO a una BOMBA: la BOMBA gana (excepto Rango 3 que destruye la bomba).

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

3. ESPECIFICACIONES DE LA INTERFAZ GRÁFICA (GUI)

Toda la aplicación deberá implementarse en Java Swing. Ningún formulario o funcionalidad del proyecto deberá depender de entrada por consola. A continuación se describen las pantallas y formularios requeridos con su comportamiento esperado. El diseño visual queda a criterio del grupo, pero deberá ser coherente, limpio y usable.

★ **IMPORTANTE:** TODAS las ventanas secundarias (excepto el Menú Inicial) deben liberarse con DISPOSE al cerrar la X. El Menú Inicial usa EXIT_ON_CLOSE.

3.1 Menú Inicial

Es la primera ventana que aparece al ejecutar el programa. Contiene únicamente:

- Botón LOG IN
- Botón CREAR PLAYER

3.1.1 LOG IN

Solicita mediante campos de texto el username y el password del jugador. Si las credenciales son válidas, se despliega el Menú Principal y se cierra (dispose) el Menú Inicial. Si son inválidas, se muestra un mensaje de error descriptivo y se permite reintentar.

3.1.2 CREAR PLAYER

Permite registrar un nuevo jugador con los siguientes datos y validaciones:

- Username: debe ser único en el sistema (validar contra el arreglo de datos).
- Password: exactamente 5 caracteres (ni más ni menos).
- Puntos: se inicializan en 0.
- Contador de partidas con Héroes: inicia en 0.
- Contador de partidas con Villanos: inicia en 0.

Si el username ya existe, se muestra un mensaje de error y se solicita ingresar uno nuevo. Si el usuario cancela la operación, se regresa al Menú Inicial. Al crear correctamente se genera:

- Un nuevo registro en arreglos

3.2 Menú Principal

Ventana principal del sistema tras iniciar sesión. Debe tener una barra de menú o panel de navegación con las siguientes secciones:

Sección	Sub-opciones
STRATEGO – MARVEL HEROES!	Partida Nueva

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

Sección	Sub-opciones
MI PERFIL	Log de Mis Últimos Juegos / Cambiar mi Password / Eliminar mi Cuenta
UNIVERSO MARVEL	Ranking / Batallas
CERRAR SESIÓN	(Confirmación antes de cerrar)

3.2.1 STRATEGO — Partida Nueva

Permite iniciar una partida nueva contra otro jugador registrado en el sistema. Si solo existe un jugador registrado, esta opción no está disponible. El jugador logueado escoge su bando (Héroes o Villanos) y el contrario toma el bando opuesto. Las fichas se colocan de forma aleatoria siguiendo estas reglas de formación:

7. En la última fila de formación de cada bando (filas 0 y 9): colocar aleatoriamente la pieza TIERRA (no puede estar en columna 0 ni 9). Alrededor de la TIERRA se colocan las BOMBAS.
8. Colocar las BOMBAS restantes aleatoriamente en las últimas 2 filas de cada bando (filas 0-1 y 8-9).
9. Colocar las piezas de Rango 2 aleatoriamente en las filas 2-3 y 6-7.
10. Colocar aleatoriamente las piezas de la primera fila de formación (filas 3 y 6).
11. Rellenar los espacios restantes con las piezas sobrantes de forma aleatoria.

Durante la partida la pantalla debe mostrar en todo momento:

- El tablero con las fichas (según la modalidad seleccionada).
- Panel de piezas vencidas de cada bando.
- Indicador del turno actual (Héroes o Villanos).
- Botón RETIRO (con confirmación): termina el juego, otorga 3 puntos al oponente, regresa al Menú Principal.

Al finalizar la partida se registra un log con uno de estos formatos:

- USERNAME VENCEDOR usando HÉROES/VILLANOS ha SALVADO/CAPTURADO la TIERRA! Venciendo a USERNAME PERDEDOR — FECHA
- USERNAME PERDEDOR usando HÉROES/VILLANOS ha perdido por no tener movimientos válidos disponibles ante USERNAME VENCEDOR — FECHA
- USERNAME VENCEDOR usando HÉROES/VILLANOS ha ganado ya que USERNAME PERDEDOR usando HÉROES/VILLANOS se ha retirado del juego — FECHA

3.2.5 MI PERFIL

Muestra todos los datos del usuario logueado (excepto el password). Incluye el historial de partidas del usuario (del más reciente al más viejo). Ofrece acceso a:

- Cambiar Password: solicita el password actual como verificación; valida que el nuevo tenga exactamente 5 caracteres.

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

- Eliminar mi Cuenta: solicita confirmación ingresando el password actual; si coincide, elimina al usuario del sistema, borra todos sus datos, y regresa al Menú Inicial.

3.2.6 UNIVERSO MARVEL — Ranking

Muestra todos los jugadores activos ordenados de mayor a menor puntaje. Formato: POSICIÓN — USERNAME — PUNTOS.

3.2.7 UNIVERSO MARVEL — Batallas

Muestra estadísticas globales del sistema:

- Cantidad de usuarios activos.
- Cantidad de usuarios históricos (incluyendo los eliminados).
- Cantidad de partidas jugadas.
    - Cantidad de veces que ganaron los HÉROES.
    - Cantidad de veces que ganaron los VILLANOS.

3.2.8 CERRAR SESIÓN

Solicita confirmación al usuario. Si confirma: cierra la ventana del Menú Principal y muestra el Menú Inicial.

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

4. REQUERIMIENTOS DE PROGRAMACIÓN ORIENTADA A OBJETOS

Esta sección define los conceptos de POO que el estudiante DEBE identificar, diseñar e implementar de forma apropiada en el proyecto. No se proporciona código; el análisis, el diseño de clases y la decisión de dónde aplicar cada concepto es responsabilidad exclusiva del equipo de trabajo. La correcta aplicación de estos requerimientos es obligatoria: su ausencia penaliza la nota del proyecto.

★ **IMPORTANTE:** Todos los requerimientos de esta sección son de carácter OBLIGATORIO. No aplican puntos extra pero su incumplimiento descuenta puntos de la calificación final.

REQ-01 Herencia sin Clase Abstracta

REQ 01	**Herencia (Concreta)**	Diseñar una jerarquía de clases donde al menos una clase padre concreta sea extendida por subclases concretas.

El estudiante debe analizar cuáles entidades del dominio del juego comparten atributos y comportamientos comunes y pueden modelarse con herencia simple entre clases concretas.

Preguntas guía para el análisis:

- ¿Qué atributos comunes comparten todas las fichas del juego (nombre, rango, bando, imagen, estado)?
- ¿Qué métodos comparten todas las fichas (mover, atacar, revelar, ocultar)?
- ¿Existen subtipos de fichas que heredan el comportamiento general pero lo especializan?
- ¿Cómo modelas la jerarquía entre las fichas móviles normales, las de rango especial y las fijas?

⚠ **NOTA:** *La clase padre NO debe ser abstracta en este requerimiento. Tanto la clase padre como las subclases deben poder instanciarse. Debe existir al menos un nivel de herencia visible en el diagrama de clases.*

REQ-02 Herencia con Clase Abstracta

REQ 02	**Clase Abstracta**	Al menos una clase abstracta con métodos abstractos implementados diferente en cada subclase concreta.

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

Identificar en el dominio del problema una entidad que conceptualmente existe pero que no tiene sentido instanciar directamente, sino solo a través de sus especializaciones.

Preguntas guía para el análisis:

- ¿Existe alguna categoría general (p.ej. "Pieza del juego" o "Entidad del sistema") que es demasiado genérica para instanciarse directamente?
- ¿Qué comportamiento abstracto define la clase padre que cada subclase debe implementar de forma distinta (p.ej. calcular poder de combate, tipo de movimiento permitido)?
- ¿Cuántas subclases concretas derivarán de esta clase abstracta?
- ¿Qué atributos y métodos concretos puede ya proveer la clase abstracta a sus hijos?

⚠ **NOTA:** *Debe existir al menos una clase abstracta con al menos un método abstracto. Las subclases concretas que la extienden deben implementar dicho método de forma distinta entre sí.*

REQ-03 Interfaces

REQ 03	Interfaces	Al menos dos interfaces que definan contratos de comportamiento implementados por clases del sistema.

Las interfaces permiten establecer contratos de comportamiento sin dictar implementación. El estudiante debe identificar comportamientos transversales que distintas clases del sistema deben garantizar, independientemente de su jerarquía.

Preguntas guía para el análisis:

- ¿Qué operaciones de persistencia (guardar, cargar, eliminar) son comunes a varias entidades del sistema (jugadores, partidas, configuración)?
- ¿Qué entidades del juego deben poder ser "atacables" o "movibles", independientemente de su jerarquía?
- ¿Tiene sentido una interfaz que defina el contrato de actualización de la interfaz gráfica (Actualizable, Refrescable)?
- ¿Qué clases implementarían cada interfaz? ¿Una clase puede implementar más de una?

⚠ **NOTA:** *Se requieren al menos dos (2) interfaces distintas, cada una implementada por al menos una clase del sistema. Las interfaces no deben confundirse con clases abstractas; no contienen atributos de instancia ni implementación de métodos.*

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

REQ-04 Tipos Enumerados (enum)

REQ 04	Enum	Al menos dos enumeraciones que representen conjuntos fijos de constantes del dominio del juego.

Los enumerados modelan conjuntos de valores fijos y conocidos en tiempo de compilación. Refuerzan la seguridad de tipos y eliminan "números mágicos" o cadenas hardcodeadas en el código.

Preguntas guía para el análisis:

- ¿Qué valores discretos y fijos existen en el juego? (p.ej. bandos: HÉROES / VILLANOS, resultado del combate: GANA / PIERDE / EMPATE, tipo de pieza: NORMAL / BOMBA / TIERRA).
- ¿Cuáles son los estados posibles de una partida? (p.ej. EN_CURSO, GUARDADA, TERMINADA, RETIRADA).
- ¿Qué modos de configuración existen? (p.ej. TUTORIAL, CLÁSICO).
- ¿En qué partes del código se usarían estos enumerados para tomar decisiones (switch, comparaciones)?

⚠ **NOTA:** *Se requieren al menos dos (2) enumeraciones. Deben usarse de forma significativa en la lógica del programa (toma de decisiones, asignación de valores), no solo declararse sin utilizar.*

REQ-05 Recursividad (mínimo 3 funciones)

REQ 05	Recursividad ≥ 3 funciones	Al menos tres funciones recursivas distintas, cada una resolviendo un problema diferente del dominio del juego.

Una función recursiva se llama a sí misma para resolver un problema descomponiéndolo en subproblemas más pequeños. El estudiante debe identificar al menos tres problemas en el proyecto que se presten a una solución recursiva natural y diferente entre sí.

Áreas del proyecto donde puede aplicarse recursividad:

#	Área / Problema	Descripción del problema a resolver recursivamente
R1	Validación de movimiento del Rango 2	Verificar recursivamente si el camino en línea recta está despejado, celda por celda.
R2	Búsqueda de movimientos válidos	Recorrer recursivamente el tablero o las piezas de un bando para encontrar cuáles tienen movimientos disponibles.

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

#	Área / Problema	Descripción del problema a resolver recursivamente
R3	Cálculo de estadísticas de partidas	Recorrer recursivamente la lista de logs para acumular estadísticas (victorias Héroes, victorias Villanos, total de partidas).
R4	Generación aleatoria de formación	Colocar fichas aleatoriamente siguiendo las reglas de formación de forma recursiva hasta completar todas las posiciones.
R5	Búsqueda en el ranking	Insertar recursivamente un jugador en la lista ordenada de ranking.

⚠ **NOTA:** *Las 3 funciones recursivas deben ser DISTINTAS entre sí (resolver problemas diferentes). No se acepta repetir la misma lógica con diferente nombre. Cada función debe tener claramente definida su condición base (caso base) y su llamada recursiva.*

REQ-06 Polimorfismo

REQ 06	Polimorfismo	Uso de polimorfismo por herencia y por interfaces: tratar distintos objetos de forma uniforme mediante una referencia de tipo padre/interfaz.

El polimorfismo permite que un mismo tipo de referencia (padre o interfaz) apunte a objetos de distintas subclases y que el comportamiento específico se resuelva en tiempo de ejecución (late binding). El estudiante debe diseñar su jerarquía para aprovechar esto.

Preguntas guía para el análisis:

- ¿Se puede declarar una colección (ArrayList, arreglo) de tipo "Pieza" y almacenar en ella tanto fichas normales, como bombas y tierras?
- ¿Al invocar el método atacar() o mover() sobre una referencia de tipo Pieza, se ejecuta la versión correcta de cada subclase automáticamente?
- ¿Se puede tratar de forma polimórfica las operaciones de persistencia (guardar, cargar) a través de una interfaz?
- ¿El tablero trabaja con referencias polimórficas al dibujar / actualizar sus celdas?

⚠ **NOTA:** *Debe evidenciarse el uso de polimorfismo: una referencia de tipo padre o interfaz invoca un método que tiene implementaciones diferentes en las subclases o clases que implementan la interfaz. No basta con solo tener herencia; debe usarse de forma polimórfica.*

© UNITEC — Proyecto de Programación II | Todos los derechos reservados

REQ-07 Manejo de Excepciones y Excepciones Propias (mínimo 3)

REQ 07	Excepciones ≥ 3 propias	Al menos tres excepciones personalizadas (extends Exception / RuntimeException) integradas al flujo del programa con try-catch.

El manejo robusto de errores es esencial en cualquier aplicación profesional. El programa NO DEBE terminar nunca por una excepción no controlada. El estudiante debe identificar al menos tres situaciones de error propias del dominio y modelarlas como excepciones personalizadas.

Posibles excepciones propias del dominio:

Excepción Propia	Cuándo lanzarla	Quién la captura
UsuarioDuplicadoException	Al intentar crear un jugador con username ya existente.	Capa de lógica de registro / UI Crear Player.
PasswordInvalidoException	Al ingresar un password que no tiene exactamente 5 caracteres.	Cualquier formulario que solicite password.
MovimientoInvalidoException	Al intentar mover una pieza a una celda no permitida.	Motor del juego / Controlador del tablero.
PartidaNoEncontradaException	Al intentar cargar una partida que no existe o fue eliminada.	Módulo de carga de partidas.
CredencialesInvalidasException	Al fallar el login (usuario o password incorrectos).	Formulario de Login.

⚠ **NOTA:** *Deben implementarse al menos 3 clases de excepción propia. Cada una debe lanzarse (throw) en al menos un punto del código y capturarse (catch) en el lugar apropiado. No se aceptan excepciones declaradas pero nunca utilizadas.*

¡Éxito en el proyecto! — Recuerden: el diseño y análisis son la clave del éxito.

unitec

Departamento de Ingeniería en Sistemas y Ciencias de la Computación

© UNITEC — Proyecto de Programación II | Todos los derechos reservados